MINGGU 10

PENGUJIAN PERANGKAT LUNAK

 Copyright © September 2026

Fokus Pembelajaran

1. Dasar-Dasar Pengujian PL
2. Strategi Pengujian PL
3. Pengujian Sistem
4. Testability
5. Pengujian White Box
6. Pengujian Black Box

Copyright © September 2026

1. DASAR-DASAR PENGUJIAN PL

- Pengujian perangkat lunak adalah proses menjalankan dan mengevaluasi sebuah PL secara manual maupun otomatis untuk menguji apakah PL sudah memenuhi persyaratan atau belum, atau untuk menentukan perbedaan antara hasil yang diharapkan dengan hasil sebenarnya.
- Pengujian bertujuan untuk mencari kesalahan.
- Pengujian yang baik adalah pengujian yang memiliki kemungkinan besar dalam menemukan kesalahan sebanyak mungkin dengan usaha sekecil mungkin.

Copyright © September 2026

Tujuan Pengujian

a. Menilai apakah perangkat lunak yang dikembangkan telah memenuhi kebutuhan pemakai.

b. Menilai apakah tahap pengembangan perangkat lunak telah sesuai dengan metodologi yang digunakan.

c. Membuat dokumentasi hasil pengujian yang menginformasikan kesesuaian perangkat lunak yang diuji dengan spesifikasi yang telah ditentukan.

Copyright © September 2026

2. STRATEGI PENGUJIAN PL

Karakteristik dari Strategi Pengujian PL:

1. Melakukan pengujian yang efektif untuk menghilangkan kesalahan sebelum pengujian dimulai
2. Pengujian dimulai pada tingkat komponen dan bekerja ke arah luar menuju integrasi sistem berbasis komputer secara keseluruhan
3. Teknik pengujian yang berbeda, tepat untuk pendekatan rekayasa PL yang berbeda dan dalam waktu yang berbeda pula
4. Pengujian dilakukan oleh pengembang PL dan kelompok penguji independen

Copyright © September 2026

Strategi Pengujian



Pengujian Unit

Teknik pengujian yang menggunakan jalur spesifik di dalam sebuah struktur kontrol dari komponen untuk memastikan bahwa cakupan telah lengkap dan dapat mendeteksi kesalahan secara maksimum

Pengujian Integrasi

Fokus pada perancangan dan pembangunan arsitektur PL, dan membahas isu-isu yang berkaitan dengan dua masalah yaitu verifikasi dan pembangunan program



Pengujian Validasi

Menyediakan jaminan akhir bahwa PL memenuhi semua persyaratan informasi, fungsional, perilaku, dan persyaratan kinerja

Pengujian Sistem

PL dan elemen siatem lainnya diuji secara keseluruhan sehingga fungsi sistem dapat dicapai

 Copyright © September 2026

Jenis Strategi Pengujian

1. Pengujian Unit

- Fokus pada upaya verifikasi terhadap unit terkecil dari PL, pada logika pemrosesan internal, dan struktur data.
- Potensi kesalahan ketika penanganan pengujian dievaluasi:
  1. Deskripsi kesalahan tidak dapat dimengerti
  2. Kesalahan yang dicatat tidak sesuai dengan kesalahan yang dihadapi
  3. Kondisi kesalahan menyebabkan intervensi sistem sebelum kesalahan ditangani
  4. Pengolahan kondisi-pengecualian tidak benar
  5. Deskripsi kesalahan tidak menyediakan informasi yang cukup untuk membantu menemukan penyebab kesalahan

 Copyright © September 2026

Jenis Strategi Pengujian (Lanjutan)

2. Pengujian Integrasi

- Adalah teknik sistematik untuk membangun arsitektur PL.
- Tujuannya untuk mengambil komponen yang diuji dan membangun struktur program yang telah ditentukan oleh perancangan.
- Jenis Pengujian Integrasi:

Integrasi Bottom-Up

Memulai konstruksi dan pengujian dengan modul atomik, artinya komponen-komponen di tingkat paling rendah dalam struktur program

Integrasi Top-Down

Adalah pendekatan inkremental untuk membangun arsitektur PL. Permasalahan integrasi Top-Down kebanyakan terjadi saat dibutuhkan pemrosesan di tingkat bawah hirarki agar pengujian di tingkat yang kebih tinggi memadai

Copyright © September 2026

3. Pengujian Regresi

- Adalah pengeksekusian kembali beberapa rangkaian pengujian yang telah dijalankan untuk memastikan bahwa perubahan tidak menimbulkan efek samping yang tidak diharapkan.
- Tiga kasus pada pengujian regresi:
  a. Contoh representatif dari pengujian uang akan menjalankan semua fungsi PL
  b. Tambahan pengujian yang fokus pada fungsi PL yang mungkin dipengaruhi oleh perubahan yang terjadi
  c. Pengujian fokus pada komponen-komponen PL yang telah diubah

Copyright © September 2026

4. Pengujian Asap (*Smoke Testing*)

- Adalah pendekatan pengujian integrasi yang umumnya digunakan ketika produk PL dikembangkan.
- Dirancang sebagai mekanisme untuk proyek di waktu kritis, sehingga memungkinkan PL dinilai sesering mungkin.
- Manfaat Pengujian Asap:
  a. Risiko integrasi diminimalkan
  b. Kualitas produk akhir ditingkatkan
  c. Diagnosis dari koreksi kesalahan disederhanakan
  d. Kemajuan lebih mudah untuk dinilai

Copyright © September 2026

3. PENGUJIAN SISTEM

- Adalah serangkaian pengujian yang berbeda-beda yang tujuan utamanya adalah untuk sepenuhnya mewujudkan sistem berbasis komputer.
- Tujuan lainnya adalah untuk memverifikasi bahwa semua elemen sistem telah terintegrasi dengan baik dan menjalankan fungsi yang telah ditetapkan.

Copyright © September 2026

Jenis Pengujian Sistem

1. Pengujian Pemulihan (*Recovery Testing*)

- Adalah pengujian sistem yang memaksa PL untuk gagal dalam berbagai cara dan memverifikasikan bahwa pemulihan dilakukan dengan benar.
- Sistem harus dapat mentoleransi kesalahan, yaitu kesalahan pengolahan tidak harus menyebabkan keseluruhan fungsi sistem berhenti.
- Kegagalan sistem harus diperbaiki dalam jangka waktu tertentu.
- Pemulihan otomatis: mengevaluasi re-inisialisasi, mekanisme *checkpointing*, pemulihan data, dan *restart* untuk mengetahui apakah semua berjalan dengan benar.
- Pemulihan manual: evaluasi rata-rata waktu perbaikan untuk menentukan apakah masih dalam batas yang dapat diterima.

Copyright © September 2026

Jenis Pengujian Sistem (Lanjutan)

2. Pengujian Keamanan

Mencoba untuk memverifikasi mekanisme perlindungan yang dibangun ke dalam sistem, dan melindungi dari penetrasi yang tidak benar.

3. Pengujian Kinerja

- Dirancang untuk menguji kinerja *run-time* dari PL dalam konteks sebagai sistem yang terintegrasi.
- Pengujian kinerja dilakukan di seluruh sistem dalam pengujian proses.

Copyright © September 2026

4. Pengujian Stress

- Menjalankan sistem dengan cara meminta sumber daya dalam jumlah, frekuensi atau volume yang tidak normal.
- Dilakukan untuk mengukur kestabilan dan kehandalan sebuah sistem.
- Ketika aplikasi/web akan digunakan oleh orang banyak maka dibutuhkan pengujian stress (*stress testing)*.
- Pengujian ini untuk mengetahui apakah aplikasi sudah layak untuk dijalankan atau masih perlu dikembangkan.
- Misalnya:
  - ✓ Sejumlah besar pengguna login pada saat yang bersamaan
  - ✓ Absen secara bersamaan untuk Mhs yang melakukan ujian online

 Copyright © September 2026

5. Pengujian Deployment

- Disebut juga pengujian konfigurasi, yang menjalankan PL di setiap lingkungan di mana PL tersebut beroperasi.
- Pengujian *deployment* juga memeriksa semua prosedur instalasi dan PL instalasi khusus (mis. Installer) yang akan digunakan oleh *user* dan semua dokumentasi yang akan digunakan untuk memperkenalkan PL ke *end user*.

 Copyright © September 2026

4. TESTABILITY

- *Testability* adalah kemampuan PL untuk **dapat diuji** artinya seberapa mudah sebuah program komputer untuk bisa diuji.
- Testabilitas memungkinkan penilaian yang mudah dan menentukan keseluruhan upaya yang diperlukan untuk melakukan aktivitas pengujian pada perangkat lunak, yang mencakup durasi pengujian, jumlah kasus pengujian, dan skenario.
- Semakin rendah tingkat testabilitas, semakin besar upaya yang diperlukan untuk memeriksa PL.
- Pengujian adalah tugas tanpa akhir yang berlangsung selama PL dikembangkan, diubah, dan diperbarui sesuai dengan persyaratan teknologi baru dan preferensi pengguna.

Copyright © September 2026

Karakteristik Testability PL

- Kemampuan untuk bisa dioperasikan (operability)
- Kemampuan untuk bisa diobservasi (observability)
- Kemampuan untuk dapat dikontrol (*controllability*)
- Kemampuan untuk dapat disusun (decomposability)
- Kesederhanaan (*simplicity*)
- Stabilitas (*stability*)
- Kemampuan untuk dapat dipahami (understandability)

Copyright © September 2026

5. PENGUJIAN WHITE BOX

- Disebut juga pengujian kotak kaca (***glass box testing***).
- Merupakan sebuah filosofi perancangan *test case* yang menggunakan struktur kontrol.
- *Test case* pada *white box*:
  a. Menjamin bahwa semua jalur independen di dalam modul telah dieksekusi sedikitnya satu kali
  b. Melaksanakan semua keputusan logis pada sisi benar dan salah *(decision)*
  c. Melaksanakan semua perulangan (*loop)* yang memenuhi semua batas operasional
  d. Melakukan struktur data internal untuk memastikan kebenarannya

Copyright © September 2026

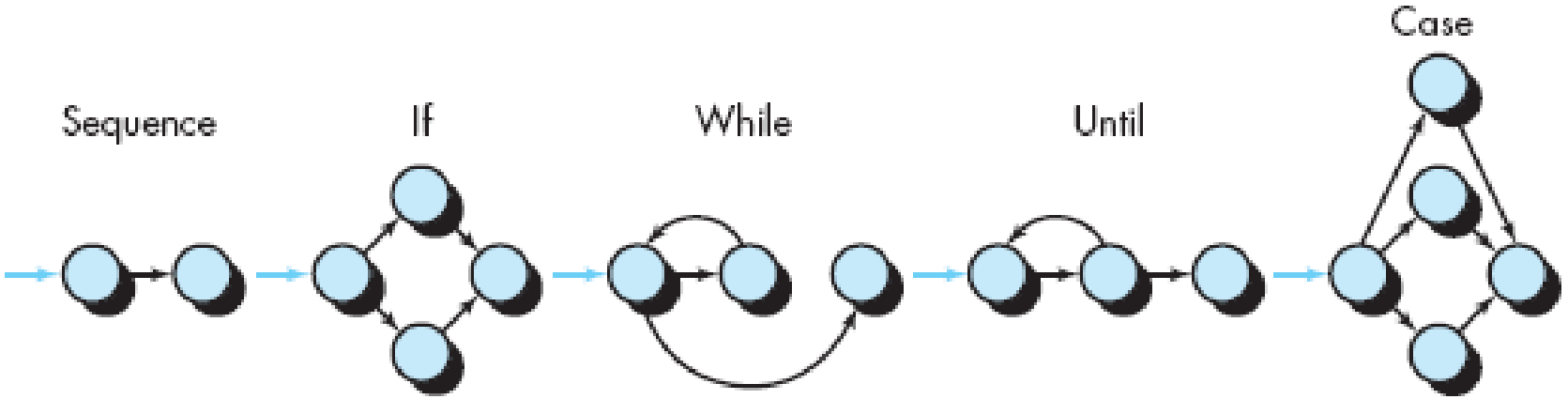
A. Pengujian Jalur Dasar (Basis Path Testing)

- Adalah teknik pengujian yang memungkinkan perancangan *test case* untuk menurunkan ukuran kompleksitas logis dari suatu rancangan prosedural dan menggunakan ukuran ini sebagai pedoman untuk menentukan rangkaian dasar jalur eksekusi.
- *Test case* diturunkan untuk menguji rangkaian dasar yang dijamin untuk mengeksekusi setiap pernyataan dalam program, setidaknya satu kali selama pengujian.
- Menggambarkan arus kontrol logis dengan menggunakan Notasi Grafik Alir (*Flow Graph*)

Copyright © September 2026

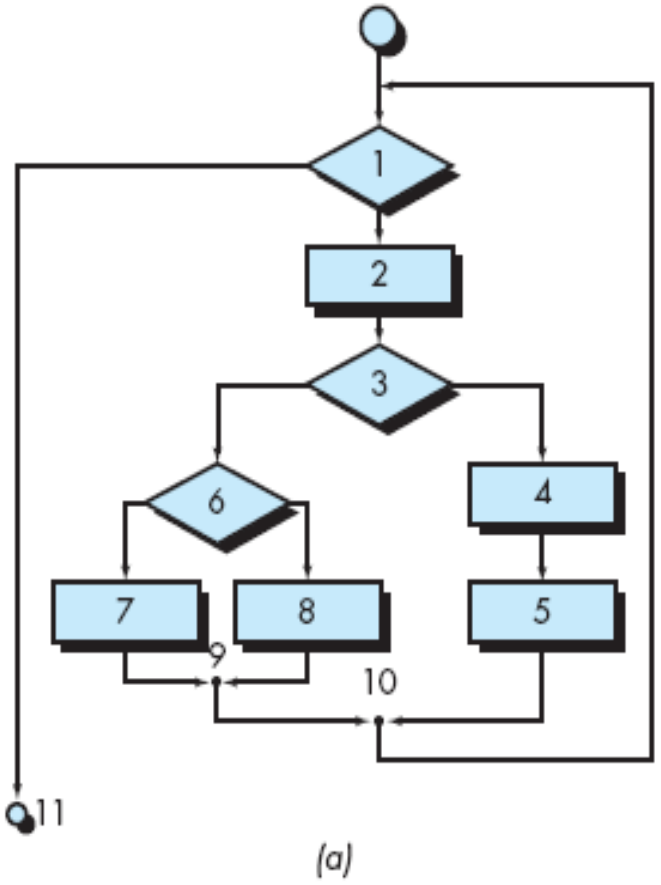
a. Notasi Grafik Alir (*Flow Graph*)

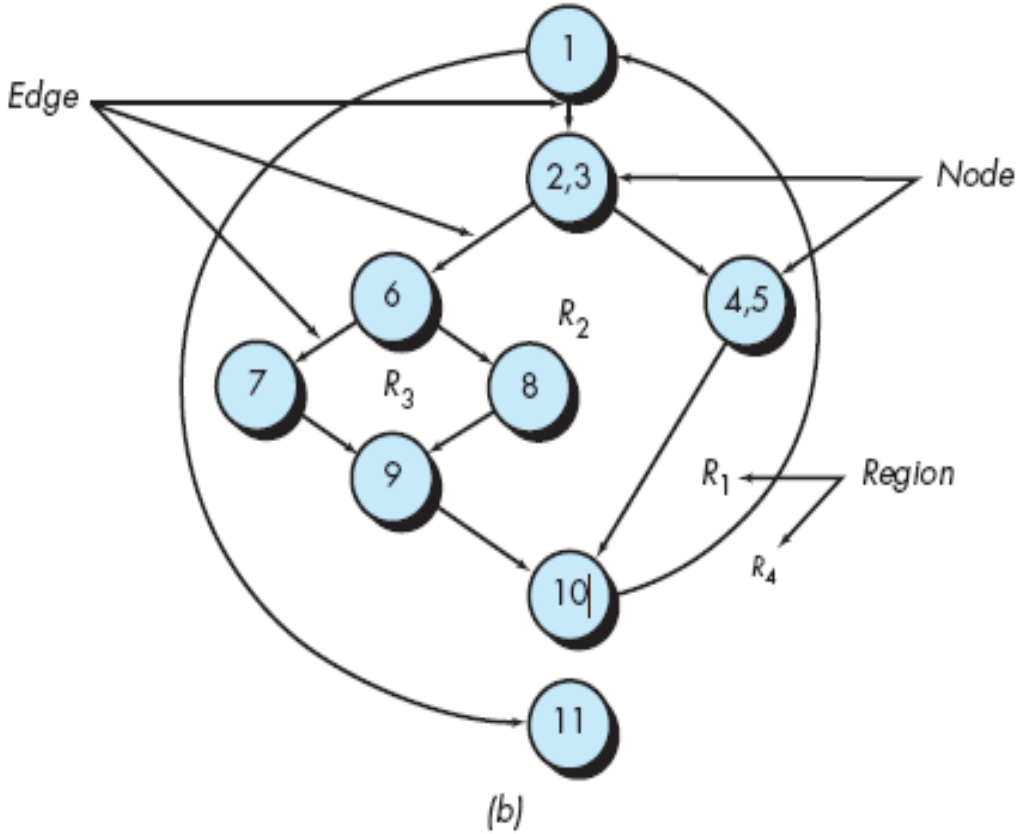
- Adalah notasi sederhana untuk merepresentasikan aliran kontrol logis.
- Lingkaran mewakili pernyataan kode program
- Notasi *flow graph* seperti gambar di bawah ini:



 Copyright © September 2026

Untuk menggambarkan *flow graph*, dengan merepresentasikan perancangan prosedural seperti gambar berikut:





Notasi Flowchart ***Flow Graph***

 Copyright © September 2026

b. Notasi *Flow Graph* (Lanjutan)

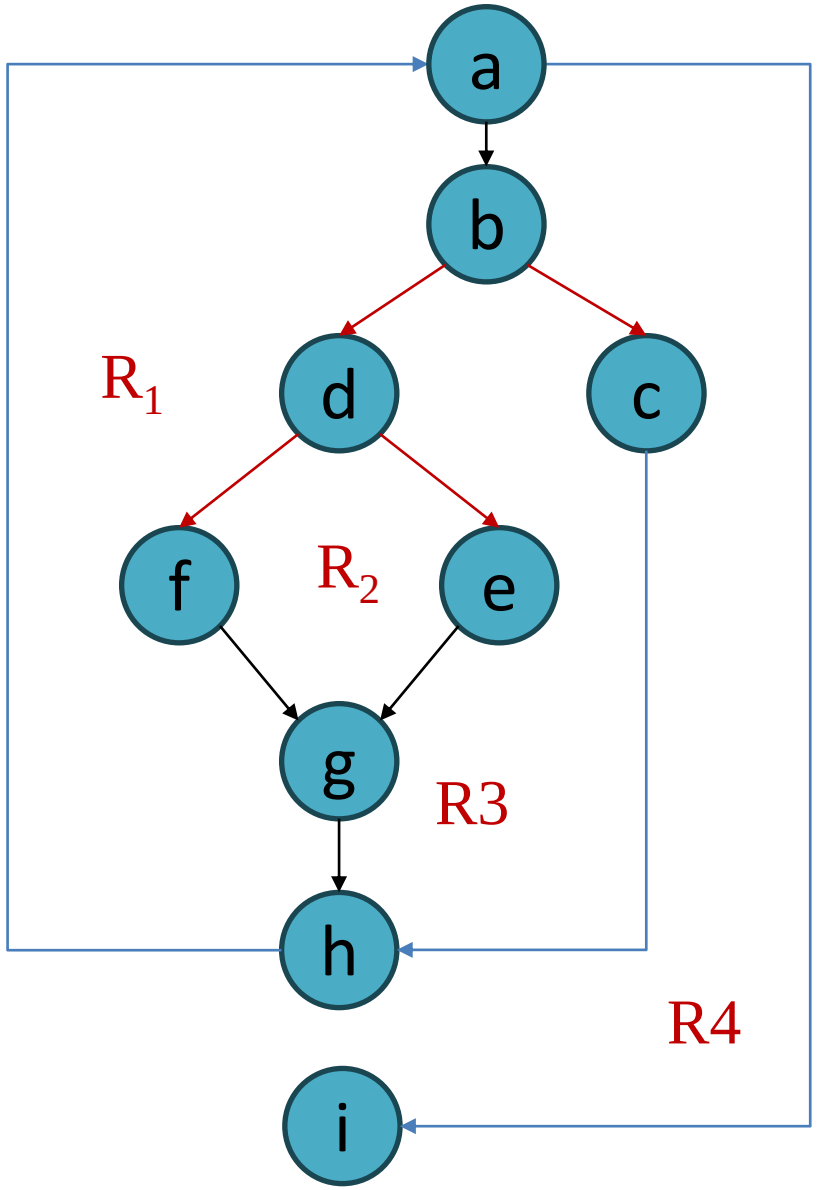
- Lingkaran menunjukkan **simpul** (***node***), merupakan satu atau lebih pernyataan-pernyataan prosedural
- Panah menunjukkan ***edge*** atau ***link***, merupakan aliran kendali
- Area yang dibatasi oleh *edge* dan *node* disebut ***region***.
- *Flow graph* menjadi rumit ketika adanya kondisi gabungan pada saat satu atau operator *boolean* ada dalam pernyataan bersyarat.
- *Node* yang berisi kondisi disebut ***node* predikat** dan ditandai oleh dua atau lebih *edge* yang berasal dari *node* tersebut.

Copyright © September 2026

Notasi Flow Graph

Node a, b dan d adalah
Node Predikat

Ada 4 Region yang merupakan daerah tertutup dari graph



 Copyright © September 2026

B. Kompleksitas Siklomatik

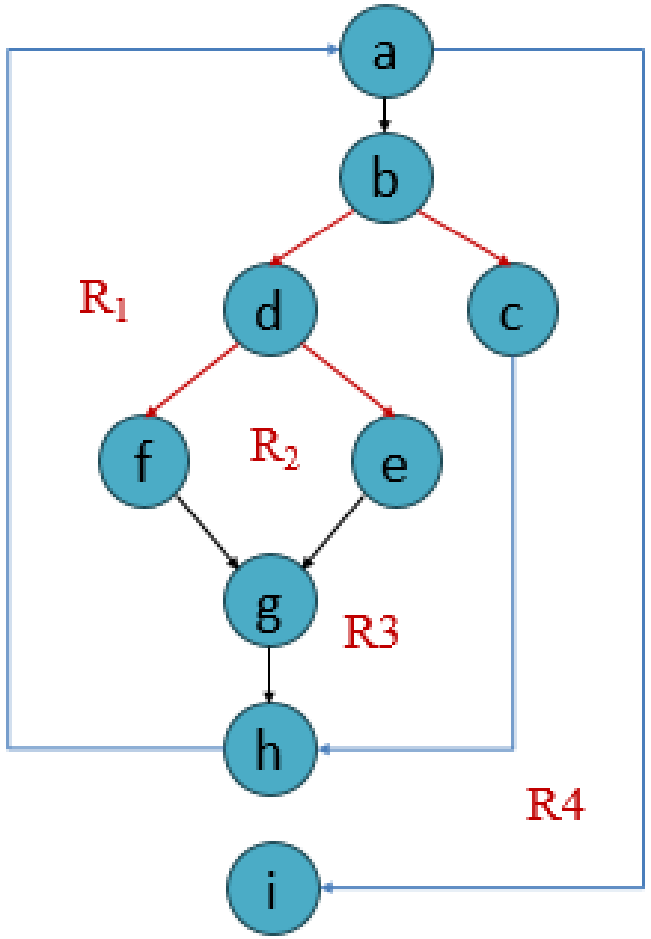
- Adalah metrik PL yang menyediakan ukuran kuantitatif dari kompleksitas logis suatu program.
- Perhitungan Kompleksitas Siklomatik:
  a. Jumlah daerah-daerah (*region*) *flow graph* yang berhubungan dengan Kompleksitas Siklomatik
  b. Kompleksitas Siklomatik V (G) = E – N + 2 dimana E adalah jumlah *edge*, N adalah jumlah *node*.
  c. Kompleksitas Siklomatik V (G) = P + 1 dimana P adalah jumlah *node* predikat.

Copyright © September 2026

Kompleksitas Siklomatik (Lanjutan)

Menghitung Siklomatik::

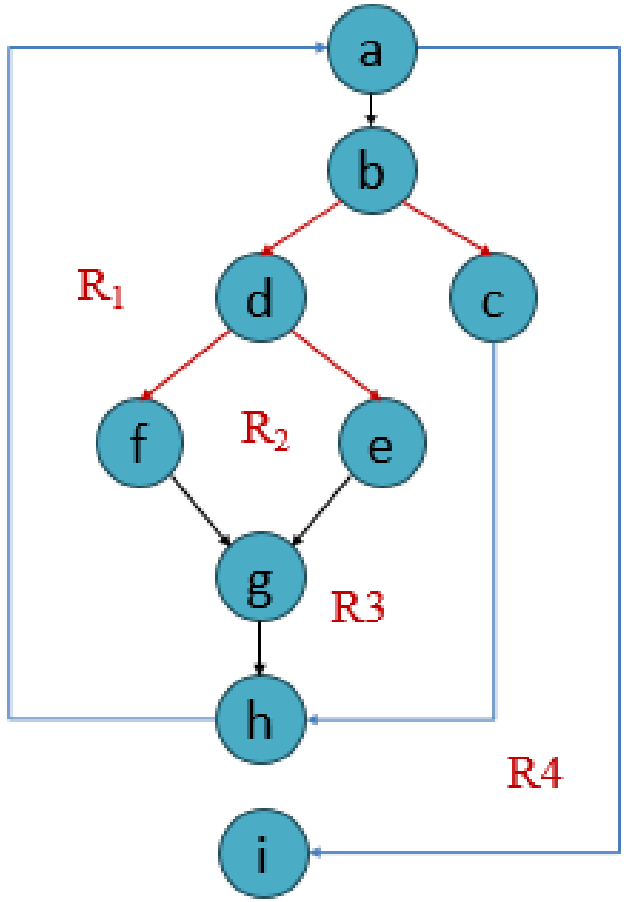
- Jumlah *region* adalah 4
- V (G) = 11 *edge* – 9 *node* + 2 = 4
- V (G) = 3 node predikat + 1 = 4
- Jadi Kompleksitas Siklomatiknya adalah 4



 Copyright © September 2026

C. Jalur Independen (Independent Path)

- Jalur Independen adalah setiap jalur yang melalui program yang memperkenalkan setidaknya satu kumpulan pernyataan-pernyataan pemrosesan atau kondisi baru.
- Jalur independen gambar disamping:
  - **Path 1**: a-i
  - **Path 2**: a-b-c-h-a-i
  - **Path 3**: a-b-d-f-g-h-a-i
  - **Path 4**: a-b-d-e-g-h-a-i
- Path 1 sampai 4 merupakan **basis set.**



 Copyright © September 2026

D. Menghasilkan *Test Case*

Pseudocode Login:

```
a   → INPUT uname+paswd
b   → IF uname+paswd valid
c   →     THEN "berhasil login"
d   →  ELSE
           ulang Login
e   → ENDIF
```

Catatan: Dosen menjelaskan langkah-langkah di atas dengan membuat *flow graph*, menghitung kompleksitas siklomatik, dan menentukan *independent path*

Copyright © September 2026

E. Pengujian Struktur Kontrol

a. Pengujian Kondisi

- Pengujian kondisi adalah metode perancangan *test case* yang menguji kondisi logis yang terdapat dalam modul program.
- Kondisi sederhana adalah variabel *boolean* atau ekspresi relasional, kemungkinan didahului oleh satu operator NOT
- Jenis kesalahan dalam kondisi meliputi kesalahan operator *boolean*, kesalahan variabel *boolean*, kesalahan kurung *boolean*, kesalahan operator relasional, dan kesalahan ekspresi aritmatika.

Copyright © September 2026

b. Pengujian Perulangan

Adalah teknik pengujian *white box* yang fokus pada validitas konstruksi perulangan.

(1) Perulangan Sederhana

Pengujian dilakukan dengan mudah, dimana n jumlah maksimum yang diijinkan melewati perulangan:

(a) Melewati perulangan secara keseluruhan

(b) Hanya satu kali melalui perulangan

(c) Dua kali melalui perulangan

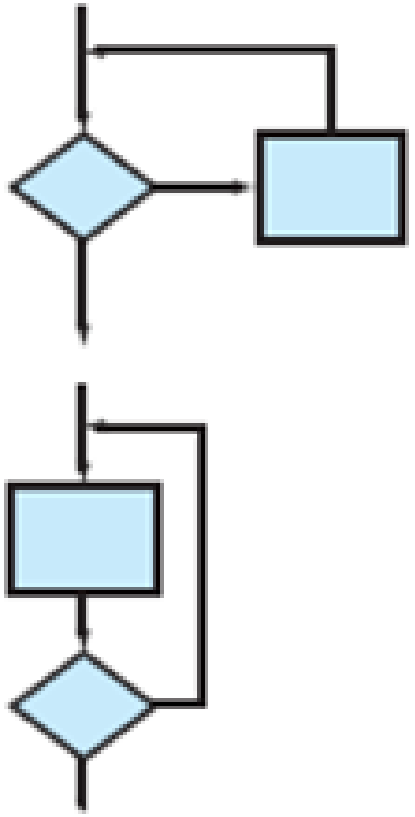
(d) Melalui perulangan sebanyak m dimana $m < n$

(e) n – 1, n, n + 1 melalui perulangan

Copyright © September 2026

Contoh Perulangan Sederhana

Berikut diberikan pseudocode tentan perulangan sederhana, dan carilah hasilnya.



```
x : integer
x = 1
WHILE (x < 5) DO
  PRINT (x)
  x = x + 1
ENDWHILE
```

 Copyright © September 2026

(2). Perulangan Bersarang

Menggunakan pendekatan perulangan sederhana, sehingga jumlah pengujian akan meningkat.

Petunjuk pengujian:

(a) Mulai dari perulangan terdalam dan atur semua perulangan ke nilai minimum

(b) Lakukan pengujian perulangan sederhana untuk perulangan terdalam, sambil menjaga perulangan luar pada nilai minimum (misal *counter* perulangan)

(c) Lanjutkan pada perulangan ke luar dan lakukan pengujian pada perulangan berikutnya

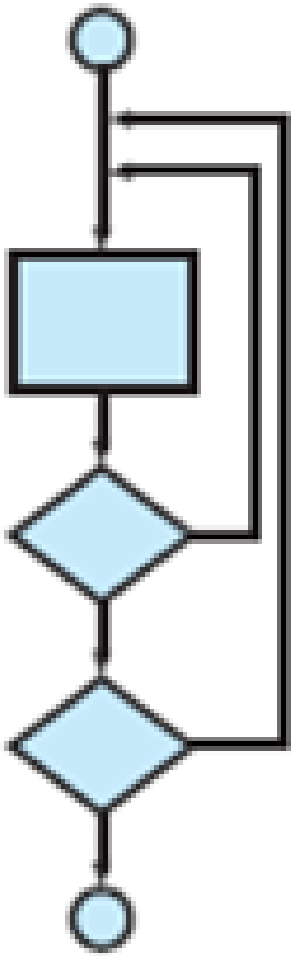
(d) Lakukan sampai semua perulangan telah diuji

 Copyright © September 2026

Contoh Perulangan Bersarang

Berikut diberikan pseudocode tentang perulangan bersarang, dan carilah hasilnya.

```
i, j : integer
FOR  i = 1  TO  4  DO
   FOR  j = 1  TO  3  DO
      PRINT (i, j)
   ENDFOR
ENDFOR
```



 Copyright © September 2026

(3). Perulangan Terangkai

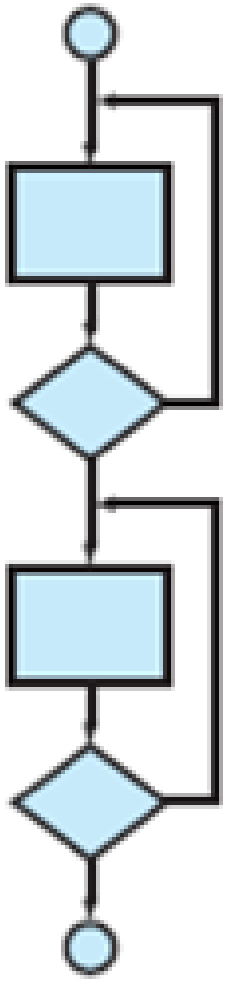
- Pengujian menggunakan pendekatan perulangan sederhana bila masing-masing perulangan independen.
- Tetapi bila dua perulangan dirangkai dan *counter* perulangan 1 digunakan sebagai harga awal perulangan 2 maka perulangan tersebut menjadi tidak independen, dan direkomendasikan ke perulangan tersarang

Copyright © September 2026

Contoh Perulangan Terangkai

```
x, y : integer
x = 0
y = 1
WHILE (x < 20) DO
   PRINT (x)
   x = x + 2
ENDWHILE

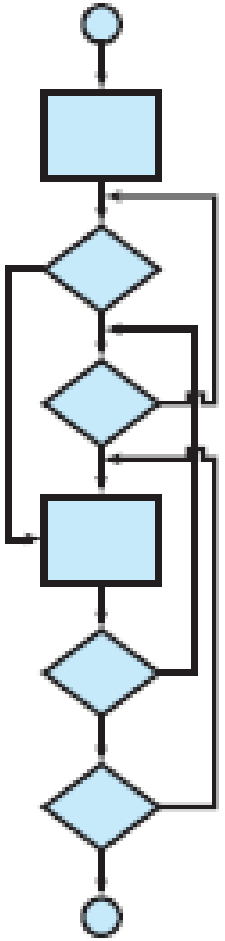
WHILE (y < 20) DO
   PRINT (y)
   y = y + 2
ENDWHILE
```



 Copyright © September 2026

(4). Perulangan Tak Terstruktur

Kapan saja memungkinkan, perulangan didisain kembali agar mencerminkan penggunaan konstruksi pemrograman terstruktur.



Copyright © September 2026

6. PENGUJIAN BLACK BOX

- Disebut juga pengujian perilaku.
- Pengujian *black box* memungkinkan untuk membuat beberapa kumpulan kondisi *input* yang akan melakukan semua kebutuhan fungsional untuk program.
- Kategori kesalahan pada pengujian *black box*:
  a. Fungsi yang salah atau hilang
  b. Kesalahan antarmuka
  c. Kesalahan struktur data atau akses basis data eksternal
  d. Kesalahan perilaku atau kinerja
  e. Kesalahan inisialisasi dan penghentian

Copyright © September 2026

1. Metode Pengujian Berbasis Grafik

Langkah-langkah pengujian:

- Memahami objek-objek yang dimodelkan dalam PL dan penghubung yang menghubungkan objek-objek tersebut
- Node direpresentasikan sebagai lingkaran.
- Hubungan direpresentasikan dengan anak panah
- Hubungan satu arah (*directed link*) bahwa hubungan bergerak hanya satu arah.
- Hubungan dua arah atau hubungan simetris (*bidirection link*) bahwa hubungan berlaku dua arah.
- Hubungan paralel digunakan ketika ada sejumlah hubungan yang berbeda yang dibangun di antara *node-node* grafik.

Copyright © September 2026

2. Partisi Kesetaraan (Equivalence Partitioning)

- Adalah metode pengujian *black box* yang membagi daerah *input* program ke dalam kelas-kelas data dari *test case* yang dapat diturunkan.
- Sebuah kelas kesetaraan merepresentasikan keadaan valid atau tidak valid dari kondisi *input.*
- Contoh: kesalahan terhadap semua data karakter yang mungkin mengharuskan banyak *test case* sebelum kesalahan umum teramati.

Copyright © September 2026

Partisi Kesetaraan (Lanjutan)

Kelas kesetaraan dapat didefinisikan:

- Jika kondisi *input* menspesifikasikan range, satu kelas kesetaraan yang valid dan dua kelas kesetaraan yang tidak valid didefinisikan
- Jika kondisi *input* membutuhkan nilai tertentu, satu kelas kesetaraan yang valid dan dua kelas kesetaraan yang tidak valid didefinisikan
- Jika kondisi *input* menspesifikasikan anggota dari himpunan, satu kelas kesetaraan yang valid dan dua kelas kesetaraan yang tidak valid didefinisikan
- Jika kondisi *input* adalah *boolean*, satu kelas kesetaraan yang valid dan dua kelas kesetaraan yang tidak valid ditentukan

Copyright © September 2026

Contoh Pengujian *Equivalence Partitioning*

Ketentuan diskon 20% untuk pembelian barang sejenis sebanyak 2, dan diskon 30% untuk pembelian barang sejenis sebanyak 3 atau lebih

No	Test Case	Hasil yang diharapkan	Hasil yang diperoleh	Ket
1.	User mengisi jumlah barang = 0	Sistem tidak memproses transaksi dan menampil-kan umpan balik	Sistem menampilkan pesan bahwa jumlah pembelian harus diisi	Berhasil
2.	User mengisi jumlah barang = 1	Sistem akan menghitung diskon dan subtotal	Sistem menghitung diskon sebesar 0 dan menghitung subtotal	Berhasil
3.	User mengisi jumlah barang = 2	Sistem akan menghitung diskon dan subtotal	Sistem menghitung diskon sebesar 20% dan menghitung subtotal	Berhasil
4.	User mengisi jumlah barang ≥ 3	Sistem akan menghitung diskon dan subtotal	Sistem menghitung diskon sebesar 30% dan menghitung subtotal	Berhasil

Copyright © September 2026

3. Analisis Nilai Batas (Boundary Value Analysis)

- Merupakan teknik perancangan *test case* yang melengkapi partisi kesetaraan dengan fokus pada kondisi *input*, dan juga akan menghasilkan *output*.
- Banyak kesalahan terjadi pada kesalahan *input*.
- BVA mengijinkan untuk menyeleksi kasus uji yang menguji batasan nilai input.
- BVA merupakan komplemen dari *equivalence partitioning*, lebih memilih pada elemen-elemen di dalam kelas ekivalen pada bagian sisi batas dari kelas

Copyright © September 2026

Pedoman BVA

a. Jika kondisi *input* menspesifikasikan range yang dibatasi oleh nilai a dan b, *test case* harus dirancang dengan nilai a dan b dan hanya di atas dan di bawah nilai a dan b

b. Jika kondisi *input* menspesifikasikan sejumlah nilai, *test case* harus dikembangkan untuk menguji jumlah-jumlah minimum dan maksimum.

c. Terapkan pedoman 1 dan 2 untuk kondisi *input*.

d. Jika struktur data program internal memiliki batas-batas yang telah ditentukan, pastikan untuk merancang *test case* untuk menguji struktur data pada batasnya.

Copyright © September 2026

Contoh Pengujian BVA

Jika sistem menampilkan jumlah stok barang adalah n, maka:

Noc	Test Case	Hasil yang diharapkan	Hasil yang diperoleh	Ket
1.	User mengisi stok = 0	Sistem tidak memproses transaksi dan memberikan umpan balik	Sistem menampilkan pesan bahwa jumlah pembelian harus diisi	Berhasil
2.	User mengisi stok = 1 sampai n	Sistem akan memproses transaksi dan mengurangi jumlah stok	Sistem menghitung subtotal dan mengurangi jumlah stok	Berhasil
3.	User mengisi stok > n	Sistem tidak memproses transaksi dan memberikan umpan balik	Sistem menampilkan pesan bahwa jumlah pembelian melebihi stok	Berhasil

Catatan: sebaiknya tidak menuliskan "**sesuai harapan**" pada kolom ini. Harapannya siapa?

Copyright © September 2026

4. Pengujian Larik Ortogonal

- Dapat diterapkan untuk masalah-masalah dimana *input domain* relatif kecil tapi terlalu besar untuk mengakomodasi pengujian yang lengkap.
- Bermanfaat dalam menemukan kesalahan yang terkait dengan logika yang salah dalam komponen PL

Copyright © September 2026

Contoh:



Jika rancangan aplikasi untuk login seperti gambar di atas, maka *black box testing* dapat dibuat berdasarkan beberapa kondisi input (4 kondisi)

 Copyright © September 2026

Contoh pengujian keamanan pada User Login

No	Test Case	Hasil yang diharapkan	Hasil yang diperoleh	Ket
1.	User tidak mengisi *username* dan *password*	User tidak bisa masuk ke dalam sistem	Sistem tetap pada form Login dan menampilkan pesan untuk mengisi *username* dan *password*	Berhasil
2.	User salah mengisi *username* atau salah mengisi *password*	User tidak bisa masuk ke dalam sistem	Sistem akan menampilkan pesan jika *username* atau *password* salah	Berhasil
3.	User salah mengisi *username* dan *password*	User tidak bisa masuk ke dalam sistem	Sistem akan menampilkan pesan jika *username* atau *password* salah	Berhasil
4.	User mengisi *username* dan *password* dengan benar	User dapat masuk ke dalam sistem	Sistem akan memproses *username* dan *password* kemudian mengijinkan User untuk masuk ke sistem	Berhasil

Copyright © September 2026

Latihan

- Dosen diminta untuk membuat sebuah latihan tentang White Box Testing dan Black Box Testing.
- Contoh kasus ditentukan sendiri oleh dosen

Copyright © September 2026

Tugas Mandiri

Buat deskripsi black box testing jika diberikan gambar berikut:



 Copyright © September 2026